

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC BÁCH KHOA  
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**NHẬP MÔN TRÍ TUỆ NHÂN TẠO (CO3061)**

---

**BÀI TẬP LỚN**

**Đề tài: Hệ thống điều phối xe cứu thương**

---

Nhóm: 1

GVHD: TS. Trương Vĩnh Lân

**Sinh viên**

| Họ và tên          | MSSV    |
|--------------------|---------|
| Hồ Đắc Minh Phương | 2312738 |
| Nguyễn Đăng Khánh  | 2311512 |
| Huỳnh Hoàng Anh    | 2410078 |
| Nguyễn Anh Quân    | 2412899 |

TP. HỒ CHÍ MINH, THÁNG 4/2026

## MỨC ĐỘ ĐÓNG GÓP

Lớp: A01

Nhóm: 1

| Họ và tên          | MSSV    | Công việc được phân công                | Đóng góp | Ký tên | Ghi chú     |
|--------------------|---------|-----------------------------------------|----------|--------|-------------|
| Hồ Đắc Minh Phương | 2312738 | Mô hình hoá trạng thái và tìm kiếm      | 100%     |        |             |
| Nguyễn Đăng Khánh  | 2311512 | Mô hình hoá yếu tố không chắc chắn      | 100%     |        |             |
| Huỳnh Hoàng Anh    | 2410078 | Học máy                                 | 100%     |        |             |
| Nguyễn Anh Quân    | 2412899 | Hệ dựa trên tri thức; hệ thống tích hợp | 100%     |        | Nhóm trưởng |

# Contents

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Mô tả bài toán thực tế</b>                                      | <b>1</b>  |
| <b>2</b> | <b>Cơ sở Lý thuyết và Mô hình hóa AI</b>                           | <b>1</b>  |
| 2.1      | Mô hình học máy (Machine Learning)                                 | 1         |
| 2.2      | Hệ cơ sở tri thức và suy luận Logic (Knowledge-Based System)       | 2         |
| 2.3      | Mô hình hóa yếu tố không chắc chắn (Uncertainty Modeling)          | 3         |
| 2.4      | Biểu diễn trạng thái và không gian tìm kiếm (State & Space Search) | 3         |
| <b>3</b> | <b>Chi tiết hệ thống và các thuật toán sử dụng</b>                 | <b>4</b>  |
| 3.1      | Naive Bayes để chẩn đoán bệnh                                      | 4         |
| 3.2      | Suy luận logic để so sánh độ ưu tiên                               | 5         |
| 3.3      | Mạng Bayes để dự báo tình trạng giao thông                         | 6         |
| 3.4      | Thuật toán $A^*$ và hàm heuristic để tìm đường                     | 8         |
| 3.5      | Kiến trúc hệ thống và luồng thực thi                               | 10        |
| <b>4</b> | <b>Thực nghiệm và phân tích kết quả</b>                            | <b>12</b> |
| 4.1      | Chẩn đoán bệnh                                                     | 12        |
| 4.2      | Ưu tiên bệnh nhân                                                  | 13        |
| 4.3      | Dự báo tình trạng giao thông                                       | 14        |
| 4.4      | Bản đồ lộ trình và so sánh chi phí                                 | 15        |
| 4.5      | Hệ thống tích hợp                                                  | 16        |
| <b>5</b> | <b>Kết luận</b>                                                    | <b>18</b> |

## 1 Mô tả bài toán thực tế

Trong các tình huống cấp cứu y tế, thời gian phản ứng đóng vai trò quyết định đến khả năng sống sót của bệnh nhân. Tuy nhiên, các hệ thống điều phối xe cứu thương truyền thống thường gặp khó khăn khi đối mặt với kịch bản có nhiều yêu cầu đồng thời.

Trong tình huống một xe cứu thương phải phục vụ nhiều bệnh nhân cùng lúc, bài toán đặt ra là cần một cơ chế điều phối khoa học thay vì chỉ dựa vào thứ tự cuộc gọi. Để giải quyết bài toán trên, đề tài này đề xuất một quy trình xử lý tích hợp dựa trên AI:

- **Dự báo và chẩn đoán:** Sử dụng học máy để thực hiện chẩn đoán sơ bộ tình trạng bệnh nhân và áp dụng mạng Bayes để dự báo rủi ro giao thông trên mạng lưới đường phố.
- **Phân tầng ưu tiên:** Áp dụng suy luận logic để xếp hạng mức độ khẩn cấp của các ca bệnh dựa trên sự kết hợp giữa triệu chứng lâm sàng và mức độ nguy hiểm của bệnh lý dự đoán.
- **Tối ưu hóa lộ trình:** Sử dụng thuật toán  $A^*$  để tính toán tuyến đường hiệu quả nhất trên đồ thị giao thông động đã được cập nhật trọng số kẹt xe thực tế, giúp tối thiểu hóa thời gian di chuyển.

Mã nguồn của đề tài: [GitHub repository](#)

Notebook của đề tài: [Google Colab notebook](#)

## 2 Cơ sở Lý thuyết và Mô hình hóa AI

### 2.1 Mô hình học máy (Machine Learning)

Trong các tình huống khẩn cấp, thông tin thu thập từ tổng đài thường là các mô tả rời rạc về triệu chứng. Nhiệm vụ đầu tiên của hệ thống là ánh xạ tập hợp các triệu chứng thô này thành một chẩn đoán bệnh lý sơ bộ, làm cơ sở cho các quyết định ưu tiên tiếp theo.

#### 2.1.1 Mô hình hóa bài toán

Bài toán được định hình dưới dạng phân loại đa lớp có giám sát (supervised multiclass classification):

- Gọi  $\mathbf{X} = \{x_1, x_2, \dots, x_n\}$  là vector đặc trưng đại diện cho tập hợp toàn bộ các triệu chứng trong cơ sở dữ liệu. Mỗi phần tử  $x_i \in \{0, 1\}$  là một biến nhị phân, thể hiện sự vắng mặt (0) hoặc xuất hiện (1) của triệu chứng thứ  $i$ . Để tối ưu hóa tài nguyên bộ nhớ khi làm việc với bộ dữ liệu lớn,  $\mathbf{X}$  được biểu diễn dưới dạng ma trận thưa.
- Gọi  $y \in \{C_1, C_2, \dots, C_k\}$  là tập hợp các nhãn bệnh lý cần dự đoán.

#### 2.1.2 Ứng dụng thuật toán Bernoulli Naive Bayes

Do không gian đặc trưng mang tính chất nhị phân phân biệt, thuật toán Bernoulli Naive Bayes được lựa chọn làm mô hình cốt lõi. Mô hình này vận hành dựa trên giả định tính độc lập có điều kiện: sự xuất hiện của một triệu chứng là hoàn toàn độc lập với các triệu chứng khác khi đã biết trước nhãn bệnh lý.

Định lý Bayes cho bài toán chẩn đoán được phát biểu như sau:

$$P(y|\mathbf{X}) = \frac{P(\mathbf{X}|y)P(y)}{P(\mathbf{X})} \quad (1)$$

Trong đó,  $P(y|\mathbf{X})$  là xác suất hậu nghiệm (Posterior),  $P(y)$  là xác suất tiên nghiệm (Prior), và  $P(\mathbf{X}|y)$  là độ khả dĩ (Likelihood). Nhờ giả định độc lập, độ khả dĩ tổng thể được phân rã thành tích của các xác suất biên:

$$P(\mathbf{X}|y) = \prod_{i=1}^n P(x_i|y) \quad (2)$$

Nhằm triệt tiêu mẫu số  $P(\mathbf{X})$  (vốn là hằng số với mọi lớp) và ngăn chặn rủi ro sai số dấu phẩy động khi nhân nhiều xác suất nhỏ, hệ thống đưa ra quyết định phân loại  $\hat{y}$  dựa trên quy tắc Maximum A Posteriori (MAP) kết hợp logarit hóa:

$$\hat{y} = \arg \max_y \left( \log P(y) + \sum_{i=1}^n \log P(x_i|y) \right) \quad (3)$$

## 2.2 Hệ cơ sở tri thức và suy luận Logic (Knowledge-Based System)

Việc phân loại bệnh lý chỉ cung cấp trạng thái tĩnh của bệnh nhân. Để ra quyết định điều phối xe cứu thương, hệ thống cần một cơ chế so sánh. Nhóm đề xuất xây dựng một mô-đun suy luận logic nhằm xếp hạng mức độ khẩn cấp dựa trên nguyên tắc: *Ưu tiên triệu chứng nghiêm trọng nhất; nếu tương đương, xét đến mức độ nguy hiểm của bệnh lý.*

### 2.2.1 Hệ cơ sở tri thức

Các dữ kiện thực tế (facts) được định nghĩa thông qua các vị từ logic để phân tầng mức độ nghiêm trọng (tier) từ 1 (khẩn cấp nhất) đến 5 (nhẹ nhất):

- $\text{symptom\_tier}(S, T)$ : Triệu chứng  $S$  được xếp vào nhóm độ nghiêm trọng  $T$ .
- $\text{disease\_tier}(D, T)$ : Bệnh lý  $D$  được xếp vào nhóm độ nghiêm trọng  $T$ .

### 2.2.2 Tập luật đánh giá độ nghiêm trọng

Hệ thống thiết lập các luật nội suy để so sánh thứ bậc giữa các tier và các triệu chứng/bệnh lý. Tính chất bắc cầu được áp dụng triệt để:

$$\text{higher\_tier}(T_1, T_3) \leftarrow \text{higher\_tier}(T_1, T_2) \wedge \text{higher\_tier}(T_2, T_3) \quad (4)$$

Luật so sánh trực tiếp hai triệu chứng ( $S_1, S_2$ ) và hai bệnh lý ( $D_1, D_2$ ):

$$\begin{aligned} \text{higher\_symptom\_severity}(S_1, S_2) &\leftarrow \text{symptom\_tier}(S_1, T_1) \wedge \text{symptom\_tier}(S_2, T_2) \\ &\quad \wedge \text{higher\_tier}(T_1, T_2) \end{aligned} \quad (5)$$

$$\begin{aligned} \text{higher\_disease\_severity}(D_1, D_2) &\leftarrow \text{disease\_tier}(D_1, T_1) \wedge \text{disease\_tier}(D_2, T_2) \\ &\quad \wedge \text{higher\_tier}(T_1, T_2) \end{aligned} \quad (6)$$

Để tìm ra nguyên nhân gây nguy hiểm nhất cho một bệnh nhân cụ thể  $P$ , hệ thống dùng logic phủ định để trích xuất triệu chứng nghiêm trọng nhất:

$$\begin{aligned} \text{exist\_higher\_symptom}(P, S) &\leftarrow \text{symptom}(P, S) \wedge \text{symptom}(P, S_0) \\ &\quad \wedge \text{higher\_symptom\_severity}(S_0, S) \end{aligned} \quad (7)$$

$$\text{highest\_symptom}(P, S) \leftarrow \text{symptom}(P, S) \wedge \neg \text{exist\_higher\_symptom}(P, S) \quad (8)$$

### 2.2.3 Tập luật ra quyết định ưu tiên

Dựa trên các nền tảng trên, luật ưu tiên cấp cứu cấp cao nhất ( $\text{higher\_priority}$ ) giữa hai bệnh nhân  $P_1$  và  $P_2$  được kích hoạt qua hai trường hợp:

$$\begin{aligned} \text{higher\_priority}(P_1, P_2) &\leftarrow \text{highest\_symptom}(P_1, S_1) \wedge \text{highest\_symptom}(P_2, S_2) \\ &\quad \wedge \text{higher\_symptom\_severity}(S_1, S_2) \end{aligned} \quad (9)$$

$$\begin{aligned} \text{higher\_priority}(P_1, P_2) &\leftarrow \text{highest\_symptom}(P_1, S_1) \wedge \text{highest\_symptom}(P_2, S_2) \\ &\quad \wedge \text{same\_symptom\_severity}(S_1, S_2) \\ &\quad \wedge \text{disease}(P_1, D_1) \wedge \text{disease}(P_2, D_2) \\ &\quad \wedge \text{higher\_disease\_severity}(D_1, D_2) \end{aligned} \quad (10)$$

Hệ thống truy xuất ra bệnh nhân nằm ở đỉnh của chuỗi ưu tiên (không bị ai khác ưu tiên hơn):

$$\text{highest\_priority}(P) \leftarrow \text{patient}(P) \wedge \text{patient}(P_0) \wedge \neg \text{higher\_priority}(P_0, P) \quad (11)$$

## 2.3 Mô hình hóa yếu tố không chắc chắn (Uncertainty Modeling)

Hệ thống sử dụng mô hình Mạng Bayes để xây dựng hệ thống dự báo mức độ kẹt xe dựa trên các yếu tố bối cảnh thực tế. Một mạng Bayes được biểu diễn dưới dạng:

$$\mathcal{B} = (G, \Theta)$$

Trong đó:

- $G = (V, E)$  là cấu trúc đồ thị có hướng không chu trình.
- $\Theta$  là tập các bảng phân phối xác suất có điều kiện (Conditional Probability Table - CPT).

Trong bài toán dự báo giao thông, biến mục tiêu của hệ thống là:

$$\text{traffic\_density\_level} \in \{0, 1, 2, 3\}$$

Tương ứng với 4 mức độ mật độ giao thông: "Low", "Medium", "High" và "Very High".

Biến mục tiêu này phụ thuộc trực tiếp vào bốn biến đầu vào mô tả điều kiện giao thông và môi trường:

- **Thời gian trong ngày** (`time_of_day`): "Morning Peak", "Afternoon", "Evening Peak", "Night".
- **Ngày trong tuần** (`day_of_week`): "Weekday", "Weekend".
- **Điều kiện thời tiết** (`weather_condition`): "Clear", "Rain", "Fog", "Heatwave".
- **Loại đường** (`road_type`): "highway", "main road", "inner road".

Từ đó, cấu trúc mạng Bayes được xây dựng với bốn nút cha hướng trực tiếp tới nút mục tiêu:

$$(\text{time\_of\_day}, \text{day\_of\_week}, \text{weather\_condition}, \text{road\_type}) \rightarrow \text{traffic\_density\_level}$$

Ý nghĩa của mô hình là xác suất xảy ra của từng mức độ kẹt xe sẽ thay đổi tùy thuộc vào tổ hợp các điều kiện đầu vào. Thay vì đưa ra một kết quả xác định tuyệt đối, mô hình cho phép biểu diễn mức độ bất định thông qua phân phối xác suất hậu nghiệm:

$$P(T \mid E_1, E_2, E_3, E_4)$$

Trong đó:

- $T$  là biến mật độ giao thông.
- $E_1, E_2, E_3, E_4$  lần lượt là các biến bằng chứng `time_of_day`, `day_of_week`, `weather_condition`, `road_type`.

Nền tảng toán học của mô hình dựa trên định lý Bayes:

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$

Ngoài ra, nhờ giả thiết độc lập có điều kiện giữa các biến trong DAG, phân phối xác suất đồng thời của toàn hệ thống được phân rã thành tích các phân phối cục bộ:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

Việc phân rã này giúp giảm đáng kể số lượng tham số cần lưu trữ và tăng hiệu quả tính toán so với việc lưu toàn bộ phân phối xác suất đồng thời.

## 2.4 Biểu diễn trạng thái và không gian tìm kiếm (State & Space Search)

### 2.4.1 Phát biểu bài toán tìm kiếm

Sau khi hệ thống suy luận logic đã khoanh vùng được bệnh nhân cần ưu tiên cấp cứu nhất, bài toán điều phối được quy về dạng bài toán tìm kiếm đường đi ngắn nhất. Xe cứu thương cần lập một lộ trình di chuyển tối ưu trên bản đồ đô thị thực tế (khu vực Delhi, India) để tiếp cận mục tiêu trong thời gian nhanh nhất, có xét đến các rủi ro kẹt xe.

## 2.4.2 Định nghĩa không gian trạng thái

Môi trường tìm kiếm được hình thức hóa một cách chặt chẽ thông qua cấu trúc Tuple  $\langle S, A, G, C \rangle$ :

- **Tập trạng thái ( $S$  - State Space):** Tập hợp toàn bộ các nút (Nodes) trên mạng lưới đồ thị OpenStreetMap, đại diện cho các giao lộ hoặc điểm dừng hợp lệ mà xe cứu thương có thể đi qua.
- **Tập hành động ( $A$  - Actions):** Tập hợp các cung (Edges) nối giữa các nút, đại diện cho việc xe cứu thương di chuyển từ giao lộ hiện tại sang giao lộ liền kề. Không gian hành động bị ràng buộc bởi các quy tắc giao thông (ví dụ: đường một chiều).
- **Trạng thái đích ( $G$  - Goal Test):** Hàm kiểm tra điều kiện dừng. Trạng thái đạt đích khi tọa độ nút hiện tại của xe cứu thương trùng khớp với tọa độ của nút gần nhất với vị trí bệnh nhân mục tiêu.
- **Hàm chi phí ( $C$  - Path Cost):** Chi phí tích lũy để đi qua một cung. Nhằm phản ánh thời gian di chuyển thực tế thay vì chỉ đo lường cự ly vật lý, hệ thống định nghĩa hàm trọng số động kết hợp giữa độ dài đoạn đường và dự báo kẹt xe từ Mạng Bayes:

$$Weight = Length \times (1 + \alpha \times JamLevel) \quad (12)$$

Trong đó,  $\alpha$  là hệ số thực nghiệm kiểm soát mức độ trừng phạt (penalty) của yếu tố kẹt xe lên vận tốc di chuyển của xe cứu thương.

## 3 Chi tiết hệ thống và các thuật toán sử dụng

### 3.1 Naive Bayes để chẩn đoán bệnh

Để thực hiện nhiệm vụ chẩn đoán bệnh lý từ triệu chứng, lớp đối tượng `SimpleClassifier` đã được xây dựng nhằm hiện thực hóa thuật toán Bernoulli Naive Bayes từ con số không. Lớp đối tượng này được thiết kế để sử dụng chung API với các mô hình phân loại thuộc thư viện `scikit-learn` [3], giúp dễ dàng tích hợp và đối chiếu hiệu năng. Quá trình cài đặt thuật toán bao gồm các phương thức cốt lõi sau:

#### 3.1.1 Tập dữ liệu

Thực nghiệm phân loại bệnh lý được tiến hành trên bộ dữ liệu *Disease-Symptom Dataset* thu thập từ nền tảng Kaggle [5]. Mặc dù bộ dữ liệu này đã được cấu trúc với các cột đại diện cho sự vắng mặt (0) hoặc xuất hiện (1) của triệu chứng, hệ thống không nạp trực tiếp dữ liệu thô vào mô hình mà tiến hành các bước tiền xử lý nhằm tối ưu hóa hiệu năng và bộ nhớ:

- **Mã hóa nhãn (Label Encoding):** Cột mục tiêu đại diện cho các bệnh lý (dạng chuỗi) được mã hóa thành các giá trị số nguyên thông qua `LabelEncoder` để thuật toán có thể học được.
- **Ép kiểu và tối ưu hóa ma trận:** Ma trận đặc trưng được ép sang định dạng số nguyên 8-bit (`np.int8`), sau đó được chuyển đổi thành cấu trúc ma trận thưa (Sparse Matrix). Bước này giúp tiết kiệm đáng kể tài nguyên RAM khi tính toán trên tập dữ liệu chứa lượng lớn giá trị 0.
- **Phân chia tập dữ liệu:** Tập dữ liệu được phân chia ngẫu nhiên theo tỷ lệ 80% dành cho quá trình huấn luyện và 20% dành cho quá trình kiểm thử.

Thực nghiệm đo lường hiệu năng của mô hình chẩn đoán bệnh lý dựa trên thuật toán Bernoulli Naive Bayes. Hệ thống được đánh giá qua hai phiên bản cài đặt: Mô hình Scale-up sử dụng thư viện Scikit-Learn (`BernoulliNB`) và Mô hình From Scratch do nhóm tự thiết kế bằng NumPy (`SimpleClassifier`).

#### 3.1.2 Khởi tạo mô hình

Hàm `__init__` chịu trách nhiệm khởi tạo các không gian lưu trữ và tham số cơ bản, trong đó quan trọng nhất là hệ số làm trơn Laplace (Laplace smoothing) với giá trị mặc định  $\alpha = 1.0$ . Kỹ thuật này được áp dụng nhằm ngăn chặn tình trạng xác suất bằng 0 khi một tập hợp triệu chứng chưa từng xuất hiện ở một loại bệnh trong tập huấn luyện.

### 3.1.3 Huấn luyện mô hình (fit)

Hàm `fit` nhận đầu vào là tập đặc trưng  $X$  và nhãn bệnh lý  $y$ , có khả năng xử lý linh hoạt cả cấu trúc mảng Numpy và ma trận thưa. Quá trình huấn luyện diễn ra như sau:

- Hệ thống đếm tần suất xuất hiện của từng loại bệnh (`class_counts`) và từng triệu chứng tương ứng (`feature_counts`) thông qua các mảng dữ liệu.
- Tính toán xác suất tiên nghiệm dưới dạng logarit (`class_log_prior_`) dựa trên tỷ lệ đếm được trên tổng số mẫu quan sát.
- Tính toán độ khả dĩ (Log Likelihood) kết hợp cùng hệ số Laplace Smoothing để tạo ra hai mảng lưu trữ xác suất: `feature_log_prob_` (xác suất xuất hiện triệu chứng) và `feature_log_prob_neg_` (xác suất không xuất hiện triệu chứng).

### 3.1.4 Dự đoán bệnh lý (predict)

Dựa trên dữ liệu đầu vào là các triệu chứng, mô hình áp dụng định lý Bayes để dự đoán nhãn bệnh. Thay vì dùng các vòng lặp truyền thống gây tốn kém thời gian, hệ thống được thiết kế để tối ưu hóa tốc độ bằng các phép nhân ma trận đại số tuyến tính.

Cụ thể, thuật toán tính tổng của tích ma trận đầu vào  $X$  với ma trận chuyển vị của xác suất xuất hiện, và tích ma trận  $(1 - X)$  với ma trận chuyển vị của xác suất không xuất hiện:

---

**Algorithm 1** Dự đoán bệnh lý từ ma trận triệu chứng (Predict)

---

**Require:** Vector đặc trưng đầu vào  $X$ , Ma trận xác suất  $LogProb_{feature}$ , Ma trận  $LogProb_{neg}$ , Mảng xác suất tiên nghiệm  $LogPrior_{class}$ , Tập nhãn  $Classes$

**Ensure:** Nhãn bệnh lý được dự đoán  $\hat{y}$

- 1: Kiểm tra và chuyển đổi định dạng  $X$  thành ma trận 2D
  - 2:  $Term_1 \leftarrow X \times LogProb_{feature}^T$  {Tích ma trận xác suất xuất hiện}
  - 3:  $Term_2 \leftarrow (1 - X) \times LogProb_{neg}^T$  {Tích ma trận xác suất không xuất hiện}
  - 4:  $LogLikelihood \leftarrow Term_1 + Term_2 + LogPrior_{class}$  {Áp dụng định lý Bayes}
  - 5:  $Best_{idx} \leftarrow \arg \max(LogLikelihood)$  {Lọc vị trí có xác suất cao nhất}
  - 6:  $\hat{y} \leftarrow Classes[Best_{idx}]$
  - 7: **return**  $\hat{y}$
- 

### 3.1.5 Đánh giá mô hình (score)

Phương thức `score` được cài đặt để đánh giá độ chính xác (Accuracy) cơ bản của mô hình. Hàm thực hiện dự đoán dựa trên đầu vào và trả về giá trị số thực thể hiện tỷ lệ dự đoán đúng trên tổng số mẫu kiểm thử.

## 3.2 Suy luận logic để so sánh độ ưu tiên

Mô-đun `PatientPrioritizer` xác định quyền ưu tiên cấp cứu thông qua suy luận logic. Các phương thức chính của mô-đun gồm:

### 3.2.1 Khởi tạo mô hình

Hàm `__init__()`: khởi tạo đối tượng với mô hình biểu diễn và suy luận logic (hiện đang sử dụng thư viện `pyDatalog` [4]). Đồng thời khởi tạo các vị từ và luật logic cơ sở đã định nghĩa ở phần Mô hình hoá.

### 3.2.2 Nạp tri thức (load\_knowledge\_base)

Hàm `load_knowledge_base` nạp các luật logic khai báo tier cho từng triệu chứng/bệnh (thông qua vị từ `symptom_tier` và `disease_tier`) từ file tri thức. Cơ sở tri thức đang được hệ thống sử dụng là `features/knowledgebase.txt`:



```
+ symptom_tier("hurts to breath", 1)
+ symptom_tier("irregular heartbeat", 2)
...
+ disease_tier("poisoning due to ethylene glycol", 1)
+ disease_tier("peritonitis", 2)
...
```

### 3.2.3 Xác định ưu tiên (get\_most\_prioritized\_patients)

Sau khi đã được nạp cơ sở tri thức, nếu cần xác định mức độ ưu tiên trong một danh sách bệnh nhân, phương thức `get_most_prioritized_patients()` thực hiện các bước:

1. Thêm các fact cho từng bệnh nhân, bao gồm: `patient(p)`, `symptom(p, s)`, `disease(p, predicted_disease)`.
2. Thực hiện truy vấn `highest_priority(P)` để tìm các nghiệm thỏa mãn.
3. Xóa các fact đã thêm nhằm tránh ảnh hưởng đến các lần suy luận tiếp theo.
4. Trả về danh sách các bệnh nhân có mức độ ưu tiên cao nhất.

## 3.3 Mạng Bayes để dự báo tình trạng giao thông

Hệ thống được hiện thực bằng ngôn ngữ Python với sự hỗ trợ của hai thư viện chính: `networkx` [6] để quản lý cấu trúc đồ thị DAG và `numpy` [7] để xử lý tính toán xác suất và ma trận dữ liệu. Ngoài ra, hệ thống còn hỗ trợ hai cơ chế triển khai: sử dụng thư viện `pgmpy` [1] hoặc sử dụng mô hình `SimpleBayesNetwork` và thuật toán suy diễn tự hiện thực.

### 3.3.1 Tập dữ liệu

Ở đây nhóm sử dụng tập dữ liệu Delhi Traffic Dataset [8], tập dữ liệu này là về tình trạng giao thông tại vị trí nơi mà nhóm thực hiện cho dự án, Delhi.

Cụ thể, bộ dữ liệu này mô phỏng điều kiện giao thông đô thị ở Delhi, Ấn Độ. Nó mô phỏng các mối quan hệ thực tế giữa khoảng cách di chuyển, tắc nghẽn giao thông, thời tiết, loại đường và tốc độ xe.

Bộ dữ liệu sử dụng 25 khu vực thực tế và nổi tiếng của Delhi như Connaught Place, Rohini, Dwarka, Saket, Hauz Khas, Karol Bagh, Lajpat Nagar, AIIMS, IGI và các khu vực khác.

Mỗi khu vực được gán tọa độ địa lý tổng hợp sao cho:

- Các khu vực lân cận tạo ra các chuyến đi ngắn
- Các khu vực xa tạo ra các chuyến đi dài
- Khoảng cách chuyến đi thực tế lên đến khoảng 70 km

Bộ dữ liệu này đảm bảo các mối tương quan mạnh mẽ theo kiểu thực tế:

- Giờ cao điểm → lưu lượng giao thông cao hơn → tốc độ thấp hơn
- Mưa và sương mù → tốc độ giảm
- Đường cao tốc → tốc độ cao hơn
- Đường nội thành → tốc độ chậm hơn
- Quãng đường dài hơn có nhiều khả năng đi trên đường cao tốc
- Thời gian di chuyển được tính trực tiếp từ khoảng cách và tốc độ với tiếng ồn thực tế

Lý do nhóm chọn bộ dữ liệu này vì phù hợp với logic thực tế giao thông, là dữ liệu thật được khảo sát, phù hợp với địa điểm mà dự án làm.

### 3.3.2 Tiền xử lý dữ liệu

Khác với bước tiền xử lý dữ liệu thường làm trong các bài toán học máy, vì ở đây nhiệm vụ của nhóm chỉ đơn giản là mô hình hóa yếu tố không chắc chắn. Cụ thể là từ dữ liệu thực tế, thông qua suy diễn để mô hình nên đồ thị có hướng không chu trình (DAG) với các node là các bảng phân phối xác suất. Do đó ở bước tiền xử lý dữ liệu nhóm chỉ thực hiện:

- Giữ lại các cột yếu tố thời tiết ảnh hưởng đến tắc đường như `time_of_day`, `day_of_week`, `road_type`,... và xóa bỏ các cột không liên quan.
- Chọn cột `traffic_density_level` làm target. Tuy nhiên mục tiêu không phải là xác định giá trị có xác suất xảy ra nhất mà là lấy kỳ vọng của target
- Thực hiện Label Encoding cho các cột yếu tố thời tiết.
- Dùng giá trị mode điền vào giá trị Missing value (nếu có).

### 3.3.3 Biểu diễn bảng xác suất có điều kiện

Mỗi biến trong mạng được gắn với một bảng xác suất có điều kiện. Hệ thống sử dụng lớp `TabularCPT` để lưu trữ các thành phần sau:

- **variable**: tên biến.
- **variable\_card**: số lượng trạng thái của biến.
- **evidence**: danh sách các nút cha.
- **evidence\_card**: số trạng thái tương ứng của các nút cha.
- **values**: ma trận xác suất.

Toàn bộ dữ liệu xác suất được chuyển sang dạng `numpy.ndarray` [7] nhằm tối ưu tốc độ truy xuất và tính toán.

### 3.3.4 Xây dựng cấu trúc mạng Bayes

Lớp `SimpleBayesianNetwork` kế thừa từ `nx.DiGraph` của thư viện `NetworkX` [6] để xây dựng cấu trúc đồ thị có hướng. Trong quá trình thêm cạnh mới, hệ thống tự động kiểm tra tính hợp lệ của mạng nhằm đảm bảo:

- Đồ thị không chứa chu trình.
- Mỗi nút đều có CPT tương ứng.
- Tổng xác suất tại mỗi điều kiện luôn bằng 1.

Nếu một cạnh mới tạo thành chu trình, cạnh đó sẽ bị loại bỏ và hệ thống phát sinh lỗi nhằm duy trì tính chất DAG của Mạng Bayes.

### 3.3.5 Huấn luyện mô hình

Quá trình huấn luyện được thực hiện từ tập dữ liệu lịch sử giao thông ở dạng CSV. Hệ thống thiết lập các cạnh từ bốn biến đầu vào tới biến mục tiêu `traffic_density_level`, sau đó tiến hành học các bảng xác suất có điều kiện.

- Đối với phiên bản sử dụng thư viện `pgmpy` [1], mô hình sử dụng thuật toán Maximum Likelihood Estimation (MLE) để ước lượng xác suất từ dữ liệu.
- Ở phiên bản tự xây dựng, hệ thống tính xác suất bằng phương pháp thống kê tần suất xuất hiện của các tổ hợp biến thông qua thao tác `groupby` trong `Pandas`.

Để xử lý trường hợp một tổ hợp điều kiện chưa từng xuất hiện trong dữ liệu huấn luyện dẫn đến xác suất bằng 0, hệ thống áp dụng kỹ thuật Laplace Smoothing:

$$P(X_i | Parents) = \frac{N(X_i | Parents) + 1}{N(Parents) + |X|}$$

Trong đó:

- $N(X_i | Parents)$ : số lần xuất hiện của trạng thái  $X_i$ .
- $N(Parents)$ : tổng số mẫu của tổ hợp cha.
- $|X|$ : số trạng thái của biến đang xét.

### 3.3.6 Suy diễn xác suất

Khi người dùng yêu cầu dự báo giao thông, hệ thống nhận vào bốn biến bằng chứng: thời gian trong ngày ( $E1$ ), ngày trong tuần ( $E2$ ), điều kiện thời tiết ( $E3$ ), loại đường ( $E4$ ). Sau đó hệ thống thực hiện truy vấn xác suất hậu nghiệm:

$$P(T = t | E_1, E_2, E_3, E_4)$$

Đối với dữ liệu loại đường từ OpenStreetMap/OSMNX, hệ thống thực hiện bước tiền xử lý nhằm gom các loại đường chi tiết như "motorway", "primary", "residential", "trunk",... về ba nhóm chính là "highway", "main road" và "inner road" để đồng nhất với bộ dữ liệu.

Quá trình suy diễn:

- Trong phiên bản sử dụng **pgmpy**, suy diễn được thực hiện bằng thuật toán Variable Elimination.
- Đối với mô hình tự xây dựng, hệ thống sử dụng phương pháp Inference by Enumeration với các bước:
  1. Sinh toàn bộ không gian trạng thái có thể xảy ra.
  2. Lọc các trạng thái không phù hợp với evidence.
  3. Tính xác suất đồng thời cho từng trường hợp.
  4. Thực hiện marginalization để lấy xác suất của biến truy vấn.
  5. Chuẩn hóa kết quả để thu được phân phối xác suất hợp lệ.

Cuối cùng, thay vì chỉ chọn nhãn có xác suất cao nhất, hệ thống sử dụng kỳ vọng toán học để lượng hóa mức độ giao thông:

$$\mathbb{E}[T] = \sum_{i=0}^3 i \cdot P(T = i | E)$$

Giá trị đầu ra là một số thực trong đoạn  $[0, 3]$ , cho phép biểu diễn mức độ giao thông một cách liên tục và phản ánh chính xác hơn trạng thái giao thông thực tế.

## 3.4 Thuật toán $A^*$ và hàm heuristic để tìm đường

### 3.4.1 Lựa chọn bản đồ

Để xây dựng hệ thống mô phỏng mạng lưới giao thông, dự án sử dụng dữ liệu từ nguồn OpenStreetMap (OSM) tập trung vào khu vực Delhi, Ấn Độ. Quy trình xử lý và khai thác dữ liệu được thực hiện như sau:

- **Xây dựng mạng lưới:** Dữ liệu bản đồ thô được trích xuất và chuyển đổi thành cấu trúc đồ thị (Graph) với các nút (nodes) và cạnh (edges).
- **Thông số kỹ thuật:** Hệ thống đã số hóa thành công mạng lưới bao gồm 10.159 nút giao và 24.383 đoạn đường kết nối.

- **Trực quan hóa:** Mạng lưới đường sá được mô phỏng dưới dạng đồ thị giúp nhận diện trực quan cấu trúc hạ tầng tại khu vực nghiên cứu.

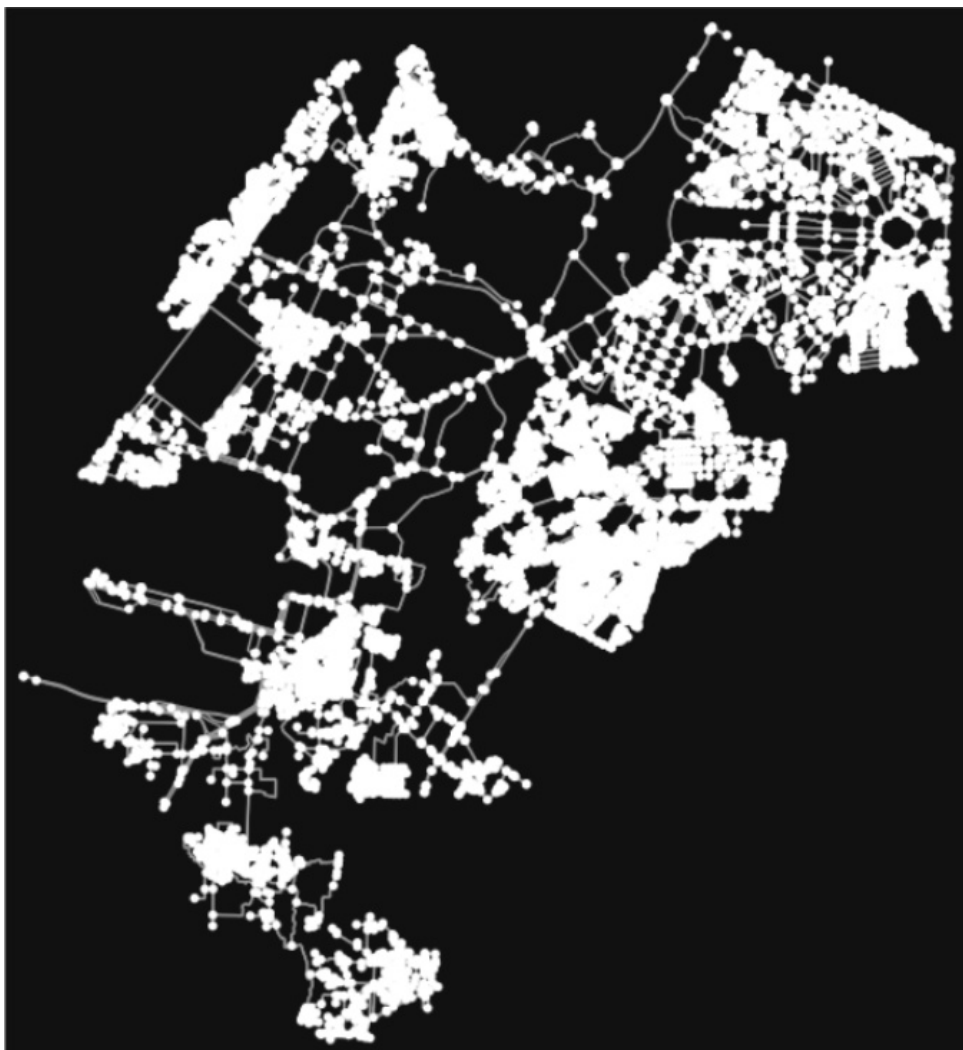


Figure 1: Hình ảnh bản đồ khu vực Delhi, Ấn Độ sau khi được trực quan hóa với các nút và các đường đi.

### 3.4.2 Thuật toán tìm kiếm $A^*$

Hệ thống sử dụng thuật toán  $A^*$  để tìm kiếm đường đi tối ưu từ vị trí xe cứu thương đến bệnh nhân. Thuật toán hoạt động bằng cách duy trì một **open\_set** (hàng đợi ưu tiên) để duyệt các nút dựa trên hàm đánh giá  $f(n) = g(n) + h(n)$ :

- $g(n)$ : Chi phí thực tế tích lũy từ điểm xuất phát đến nút hiện tại  $n$ .
- $h(n)$ : Chi phí ước lượng (heuristic) từ nút  $n$  đến đích.

Quy trình thực thi bài toán Find Optimal Path:

1. Khởi tạo **g\_score** của tất cả các nút là vô cùng, ngoại trừ điểm xuất phát bằng 0.
2. Liên tục lấy nút có **f\_score** thấp nhất từ **open\_set** để mở rộng.
3. Cập nhật hàng xóm của nút hiện tại nếu tìm thấy lộ trình có chi phí thấp hơn (**tentative\_g\_score**).
4. Lưu vết đường đi thông qua cấu trúc **came\_from** để truy hồi kết quả khi đạt đến đích.

### 3.4.3 Hàm heuristic

Hàm heuristic đóng vai trò quan trọng trong việc định hướng tìm kiếm, giúp thuật toán hội tụ nhanh hơn. Trong bài toán này, hàm heuristic được tính bằng Khoảng cách Euclid giữa tọa độ của nút hiện tại và tọa độ đích:

```
def h(node_id, target_id):
    node_coord = self.model._node[node_id]
    target_coord = self.model._node[target_id]
    return euclidean_distance(
        (node_coord['y'], node_coord['x']),
        (target_coord['y'], target_coord['x'])
    )
```

Để đảm bảo tính chính xác về mặt địa lý, khoảng cách này được tính toán thông qua thư viện `geopy` (hàm `geodesic`), đo lường khoảng cách theo mét trên bề mặt trái đất thay vì chỉ dùng tọa độ phẳng.

## 3.5 Kiến trúc hệ thống và luồng thực thi

### 3.5.1 Bộ dữ liệu

- **Disease-Symptom Dataset** [5]: bộ dữ liệu để train mô hình chẩn đoán bệnh.
- **Delhi Traffic Patterns Dataset** [8]: bộ dữ liệu để train mô hình dự báo tình trạng giao thông.
- Sử dụng dữ liệu bản đồ tại địa điểm *Delhi, India* lấy từ thư viện `OSMnx` [2].

### 3.5.2 Kiến trúc tổng quan

Các thành phần AI của hệ thống bao gồm:

- **Bản đồ và tìm kiếm:** bản đồ `MapRouter` (`map_router.py`) và thuật toán  $A^*$  (`simple_search_algorithm.py`).
- **Suy luận ưu tiên bệnh nhân:** `PatientPrioritizer` (`patient_prioritizer.py`).
- **Dự báo tình trạng giao thông:** `TrafficEstimator` (`traffic_estimator.py`).
- **Chẩn đoán bệnh:** `DiseaseClassifier` (`disease_classifier.py`).

Các thuật toán, cấu trúc dữ liệu và mô hình tự hiện thực được đặt trong `modules/models`, các thành phần AI nói trên hoạt động sử dụng các mô hình tự hiện thực, hoặc sử dụng thư viện. Ngoài ra, một đoạn script python cũng được cung cấp để hỗ trợ tương tác với hệ thống trên notebook thông qua giao diện (`dispatch-ui.py`).

### 3.5.3 Luồng thực thi

Khi thực thi, hệ thống sẽ thực hiện qua ba giai đoạn:

1. **Khởi tạo:** Khởi tạo các thành phần AI bao gồm việc tải bản đồ, các bộ dữ liệu và huấn luyện/ nạp tri thức cho các mô hình AI.
2. **Cập nhật môi trường:** Sau khi đã được huấn luyện, `TrafficEstimator` sẽ dự báo tình trạng giao thông dựa trên thời gian và loại đường và cập nhật trọng số các cạnh của bản đồ để chuẩn bị cho bước tìm kiếm.
3. **Xử lý chính:** Giả sử rằng vị trí của xe cứu thương là thông tin đã biết trước, khi đó input của hệ thống là danh sách các bệnh nhân yêu cầu xe cứu thương (kèm với thông tin về triệu chứng và vị trí của họ).

Dựa vào các triệu chứng mà bệnh nhân khai báo, hệ thống sẽ tiến hành chẩn đoán bệnh cho từng bệnh nhân. Sau đó, bệnh nhân có tình trạng khẩn cấp nhất sẽ được ưu tiên xe cứu thương và hệ thống sẽ tìm đường đi nhanh nhất tới bệnh nhân này. Nếu có nhiều bệnh nhân có cùng mức ưu

tiên cao nhất, bệnh nhân có đường đi tìm được ngắn nhất sẽ được chọn. Kết quả cuối cùng của hệ thống là đường đi tới bệnh nhân được chọn.

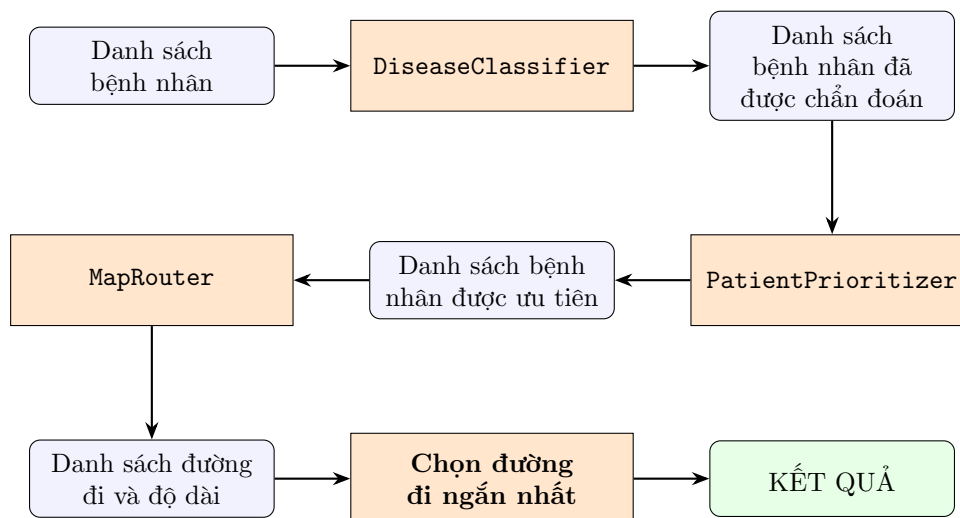


Figure 2: Sơ đồ luồng xử lý chính của hệ thống AI tích hợp

### 3.5.4 Cách chạy chương trình

Để chạy demo toàn bộ hệ thống, đầu tiên, hãy mở colab notebook của đề tài, lướt xuống phần Hệ thống tích hợp, tại đây ta có thể tùy chỉnh tham số môi trường hoặc tiếp tục với tham số mặc định:

```

time_of_day = "Night"      # "Morning Peak" / "Afternoon" / "Evening Peak" / "Night"
day_of_week = "Weekday"    # "Weekday" / "Weekend"
weather_condition = "Rain" # "Clear" / "Rain" / "Fog" / "Heatwave"
  
```

Sau khi đã tùy chỉnh cấu hình, chọn *Run All*. Sau khi toàn bộ notebook đã chạy xong, lướt xuống cell cuối cùng. Tại đây sẽ xuất hiện một giao diện để hỗ trợ tương tác với hệ thống. Lần lượt thực hiện các bước:

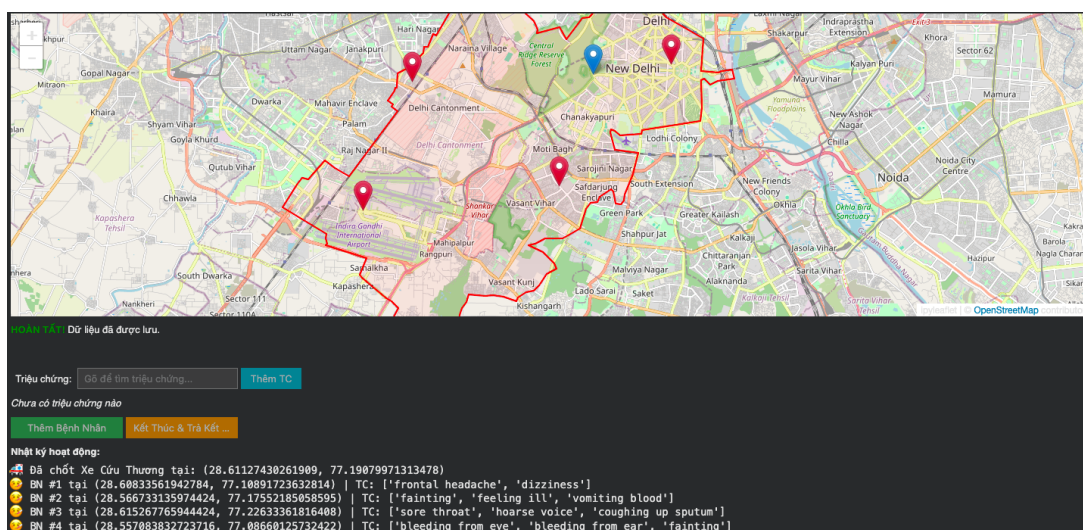


Figure 3: Tương tác với hệ thống qua giao diện trên notebook

1. Chọn vị trí xe cứu thương trên bản đồ.
2. Thêm một số lượng bệnh nhân tùy ý bằng cách lặp lại các bước sau:

- Chọn vị trí bệnh nhân trên bản đồ.
- Liệt kê các triệu chứng.
- Nhấn "Thêm bệnh nhân".

3. Nhấn "Kết thúc và trả kết quả".

**Lưu ý:** Kết quả xuất ra màn hình có bao gồm hình ảnh bản đồ, và quá trình render bản đồ có thể mất một khoảng thời gian. Do đó, vui lòng kiên nhẫn chờ đợi hình ảnh cuối cùng.

## 4 Thực nghiệm và phân tích kết quả

Phần này trình bày các kịch bản thực nghiệm nhằm đánh giá hiệu năng và tính chính xác của từng mô-đun AI đơn lẻ, cũng như khả năng phối hợp nhịp nhàng của toàn bộ đường ống (pipeline) trong hệ thống điều phối xe cứu thương.

### 4.1 Chẩn đoán bệnh

#### 4.1.1 Kết quả độ chính xác

Đánh giá trên tập dữ liệu kiểm thử (Test set) cho thấy sự đồng thuận tuyệt đối giữa hai phiên bản cài đặt. Cả hai mô hình đều đạt được các chỉ số hiệu năng tương đương nhau:

- **Accuracy (Độ chính xác):** 84.53%
- **Precision (Độ chuẩn xác):** 84.32%
- **Recall (Độ bao phủ):** 84.53%
- **F1-Score:** 83.87%

*Ví dụ minh chứng:* Khi cấp dữ liệu đầu vào gồm các triệu chứng ['anxiety and nervousness', 'dizziness'], cả hai mô hình đều đưa ra cùng một kết quả chẩn đoán chính xác là **acute stress reaction**.

#### 4.1.2 Kết quả đo lường hiệu năng

Bảng dưới đây trình bày các số liệu đo lường về thời gian xử lý và tài nguyên bộ nhớ tiêu thụ khi hệ thống thực hiện huấn luyện và xử lý 1000 truy vấn ngẫu nhiên mô phỏng (Inference Stress Test).

| Chỉ số đánh giá (Metrics)                  | Scikit-Learn (Scale-up) | NumPy (From Scratch) |
|--------------------------------------------|-------------------------|----------------------|
| Thời gian huấn luyện (sec)                 | 16.0638                 | 15.5014              |
| Bộ nhớ tối đa (MB)                         | 2783.95                 | 1696.92              |
| Thời gian suy diễn cho 1000 truy vấn (sec) | 11.4117                 | 1.2466               |

Table 1: Bảng thống kê hiệu năng phần cứng giữa hai mô hình Naive Bayes



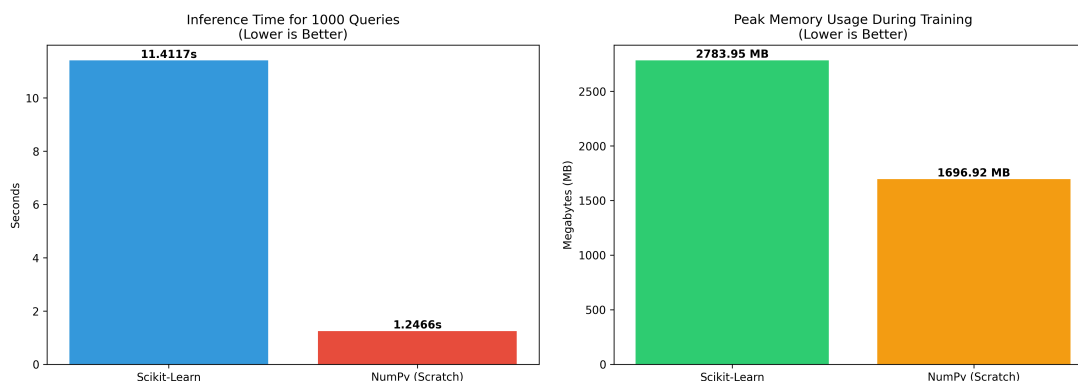


Figure 4: Biểu đồ thời gian suy diễn (trái) và bộ nhớ tiêu thụ tối đa (phải) của hai mô hình.

#### 4.1.3 Phân tích kết quả và biện luận

Dựa trên số liệu thực nghiệm, mô hình tự cài đặt (`simpleClassifierModel`) cho thấy sự vượt trội toàn diện về mặt tối ưu hệ thống khi được so sánh với thư viện chuẩn Scikit-Learn:

- **Độ chính xác tuyệt đối:** Hai phiên bản đạt sự đồng thuận 100% trên mọi chỉ số chẩn đoán, chứng minh thuật toán Naive Bayes cài đặt thủ công hoàn toàn chính xác về mặt nền tảng toán học.
- **Tối ưu tốc độ suy diễn (Nhanh hơn  $\approx 9.15$  lần):** Mô hình NumPy xử lý 1000 ca bệnh giả lập chỉ mất 1.24 giây (so với 11.41 giây của Scikit-Learn). Đột phá này có được nhờ áp dụng kỹ thuật vector hóa (Vectorization) và phép nhân ma trận trực tiếp. Trong khi đó, thư viện Scikit-Learn tốn rất nhiều tài nguyên cho các tác vụ "overhead" ngầm định như: kiểm tra tính hợp lệ của input (data sanitization), ép kiểu dữ liệu, và quản lý siêu tham số.
- **Tiết kiệm 39% RAM lưu trữ:** Bằng cách tối ưu hóa cấu trúc dữ liệu lưu trữ các tham số xác suất (Prior và Likelihood) dạng mảng thuần túy, mô hình nội bộ đã loại bỏ hoàn toàn các metadata phức tạp của object chuẩn trong Scikit-Learn. Hệ quả là bộ nhớ tiêu thụ đỉnh điểm giảm mạnh từ 2783.95 MB xuống chỉ còn 1696.92 MB.
- **Bảo toàn tốc độ huấn luyện:** Dù được tinh gọn tối đa phục vụ cho khâu suy diễn (inference), mô hình tự xây dựng vẫn duy trì tốc độ huấn luyện ngang ngửa (15.50 giây so với 16.06 giây), chứng tỏ thuật toán không hề bị đánh đổi (trade-off) hiệu suất ở giai đoạn training.

**Kết luận:** Mô hình chẩn đoán nội bộ không chỉ chính xác mà còn tối ưu xuất sắc tài nguyên phần cứng. Việc giảm thiểu độ trễ và dung lượng RAM đáp ứng hoàn hảo yêu cầu tích hợp vào các hệ thống y tế thực tế, nơi tốc độ phản hồi và giới hạn máy chủ là các yếu tố then chốt.

## 4.2 Ưu tiên bệnh nhân

### 4.2.1 Thiết lập kịch bản

Giả sử hệ thống nhận được yêu cầu cấp cứu từ 4 bệnh nhân. Danh sách này được đưa vào mô-đun `PatientPrioritizer` và lưu trong `patient_list`:

| ID | Triệu chứng                                     | Chẩn đoán                                    |
|----|-------------------------------------------------|----------------------------------------------|
| 1  | frontal headache, dizziness                     | benign paroxysmal positional vertical (bppv) |
| 2  | sore throat, hoarse voice, difficulty breathing | head and neck cancer                         |
| 3  | bleeding from eye, bleeding from ear, fainting  | hypovolemia                                  |
| 4  | fainting, feeling ill, vomiting blood           | hypovolemia                                  |

Table 2: Thông tin bệnh nhân mẫu cho mô-đun Ưu tiên



#### 4.2.2 Kết quả suy luận

Hệ thống thực thi tập luật logic thông qua lệnh:

```
most_prior_patients = patient_prioritizer.get_most_prioritized_patients(  
    patient_list)
```

Kết quả danh sách ưu tiên cao nhất thu được:

ID bệnh nhân được ưu tiên: [3, 4]

#### 4.2.3 Đánh giá và biện luận

Quá trình suy luận logic của hệ thống được diễn giải như sau:

- Bệnh nhân 1 có cả 2 triệu chứng thuộc Tier 3.
- Bệnh nhân 2 có triệu chứng nặng nhất là **difficulty breathing** thuộc Tier 1.
- Cả bệnh nhân 3 và 4 đều sở hữu các triệu chứng khẩn cấp thuộc Tier 1.

Nếu chỉ xét trên cấp độ triệu chứng, hệ thống sẽ lọc được tập ứng viên gồm bệnh nhân 2, 3 và 4. Tiếp tục nội suy mức độ nghiêm trọng của bệnh lý, cơ sở tri thức **knowledgebase.txt** ghi nhận bệnh của bệnh nhân 2 thuộc Tier 4, trong khi bệnh của bệnh nhân 3 và 4 đều thuộc Tier 1. Do đó, quyết định loại bệnh nhân 2 và giữ lại bệnh nhân 3, 4 ở mức độ ưu tiên cao nhất là hoàn toàn chính xác và tuân thủ nghiêm ngặt các luật logic đã định nghĩa.

### 4.3 Dự báo tình trạng giao thông

Để đánh giá tính chính xác và hiệu quả của hệ thống do nhóm tự phát triển (**SimpleBayesianNetwork**), chúng tôi tiến hành đối chiếu kết quả dự báo của nó với thư viện chuẩn trong ngành là **pgmpy**. Cả hai mô hình đều được huấn luyện trên cùng một bộ dữ liệu, sử dụng chung cấu trúc đồ thị và thực thi cùng các testcases.

| STT | Thời gian    | Ngày    | Thời tiết | Loại đường | SimpleBN | pgmpy | Độ lệch |
|-----|--------------|---------|-----------|------------|----------|-------|---------|
| 1   | Morning Peak | Weekday | Rain      | inner road | 1.85     | 1.91  | -0.06   |
| 2   | Night        | Weekend | Clear     | highway    | 0.70     | 0.59  | +0.11   |
| 3   | Afternoon    | Weekday | Fog       | main road  | 1.32     | 1.30  | +0.02   |
| 4   | Evening Peak | Weekend | Heatwave  | highway    | 1.59     | 1.62  | -0.03   |
| 5   | Morning Peak | Weekday | Clear     | main road  | 2.12     | 2.13  | -0.01   |
| 6   | Night        | Weekday | Rain      | inner road | 0.60     | 0.27  | +0.33   |
| 7   | Afternoon    | Weekend | Clear     | highway    | 1.06     | 1.02  | +0.04   |
| 8   | Evening Peak | Weekday | Rain      | main road  | 2.24     | 2.28  | -0.04   |
| 9   | Morning Peak | Weekend | Fog       | inner road | 1.00     | 0.50  | +0.50   |
| 10  | Night        | Weekday | Fog       | highway    | 0.91     | 0.57  | +0.34   |
| 11  | Afternoon    | Weekday | Heatwave  | inner road | 1.38     | 1.36  | +0.02   |
| 12  | Evening Peak | Weekend | Clear     | main road  | 1.44     | 1.44  | 0.00    |
| 13  | Morning Peak | Weekday | Heatwave  | highway    | 2.12     | 2.17  | -0.05   |
| 14  | Night        | Weekend | Rain      | main road  | 1.00     | 0.67  | +0.33   |
| 15  | Afternoon    | Weekend | Fog       | highway    | 1.18     | 1.00  | +0.18   |
| 16  | Evening Peak | Weekday | Heatwave  | inner road | 2.19     | 2.30  | -0.11   |
| 17  | Morning Peak | Weekend | Clear     | inner road | 1.68     | 1.70  | -0.02   |
| 18  | Night        | Weekday | Heatwave  | main road  | 0.78     | 0.57  | +0.21   |
| 19  | Afternoon    | Weekday | Rain      | highway    | 1.18     | 1.15  | +0.03   |
| 20  | Evening Peak | Weekend | Fog       | inner road | 1.27     | 1.22  | +0.05   |

**Phân tích và giải thích kết quả:**

Nhìn chung, cả hai mô hình đều bám sát quy luật giao thông thực tế và cho kết quả tương đối giống nhau ở các tình huống phổ biến như giờ cao điểm ngày thường hoặc cuối tuần trên các tuyến đường chính. Điều này cho thấy mô hình tự xây dựng đã hiện thực đúng nền tảng toán học của Mạng Bayes.

Tuy nhiên, sự khác biệt bắt đầu xuất hiện ở các tình huống hiếm gặp hoặc có ít dữ liệu lịch sử, chẳng hạn như thời tiết xấu vào ban đêm hoặc lưu lượng thấp trên đường nội bộ. Trong các trường hợp này, thư viện `pgmpy` thường đưa ra mức dự báo rất thấp do phụ thuộc mạnh vào tần suất xuất hiện trong dữ liệu huấn luyện.

Ngược lại, mô hình tự code cho kết quả ổn định và thực tế hơn nhờ áp dụng kỹ thuật Laplace Smoothing. Cơ chế này giúp hệ thống luôn duy trì một lượng xác suất nhỏ cho các trường hợp hiếm, tránh hiện tượng xác suất bằng 0 khi dữ liệu bị thiếu.

Nhờ đó, mô hình tự phát triển có khả năng thích nghi tốt hơn với các tình huống bất thường và cho thấy độ bền vững cao hơn trong môi trường giao thông thực tế, nơi luôn tồn tại nhiều biến động khó dự đoán.

## 4.4 Bản đồ lộ trình và so sánh chi phí

### 4.4.1 Hình ảnh bản đồ lộ trình

Dựa trên kết quả thực thi hệ thống với dữ liệu thực tế tại khu vực Delhi, India, lộ trình di chuyển tối ưu của xe cứu thương được mô phỏng trực quan như sau:

- **Vị trí xuất phát (Ambulance):** Được đánh dấu bằng chấm tròn xanh (Green Dot) tại tọa độ (28.6019, 77.2300).
- **Vị trí mục tiêu (Patient):** Được đánh dấu bằng chữ X đỏ tại tọa độ (28.5320, 77.1506).
- **Lộ trình tối ưu:** Đường kẻ màu đỏ thể hiện con đường ngắn nhất được tìm thấy thông qua thuật toán  $A^*$ .
- **Mật độ giao thông:** Các cạnh trên bản đồ được hiển thị với màu sắc từ xanh đến đỏ tương ứng với mức độ kẹt xe thực tế tại thời điểm truy vấn.

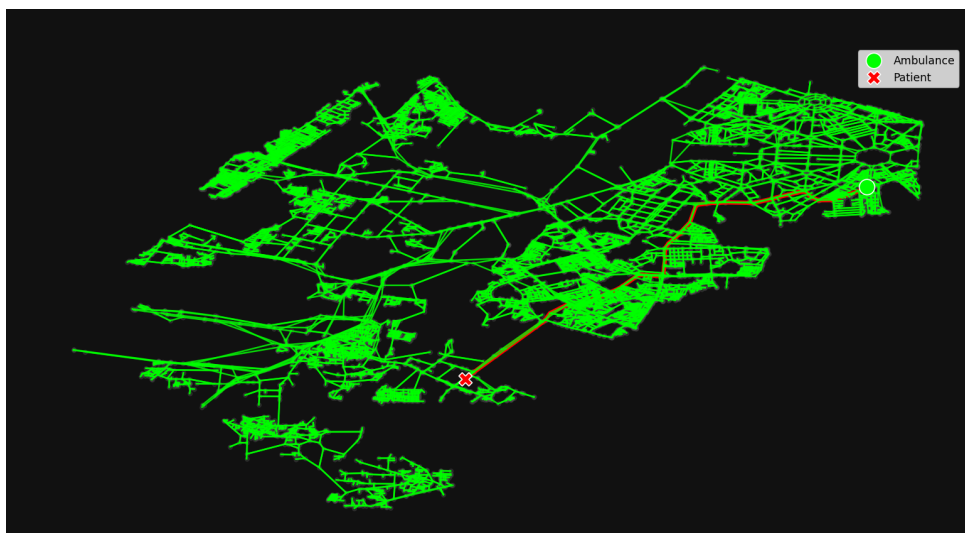


Figure 5: Lộ trình tối ưu (đường đỏ) trên đồ thị 10,160 nút giao thông tại Delhi.

### 4.4.2 Kết quả tính toán chi phí

Dựa trên kết quả thực thi, hệ thống đã thực hiện tìm kiếm trên không gian trạng thái mạng lưới đường bộ. Các thông số kỹ thuật thu được từ lộ trình tối ưu như sau:

| Chỉ số                             | Giá trị thực nghiệm                   |
|------------------------------------|---------------------------------------|
| Số lượng tọa độ trong lộ trình     | 114 điểm (waypoints)                  |
| Tổng chiều dài quãng đường ( $L$ ) | 13,141.72 meters                      |
| Thuật toán sử dụng                 | $A^*$ Search với Heuristic Euclid     |
| Trọng số tối ưu ( $Weight$ )       | Đã bao gồm hệ số trừng phạt do kẹt xe |

Table 3: Bảng thông số kết quả tìm đường thực tế.

#### 4.4.3 Biện luận và Phân tích hiệu quả

Kết quả thực nghiệm cho thấy sự hiệu quả vượt trội khi kết hợp giữa lý thuyết đồ thị và dữ liệu giao thông thực tế:

- Độ chính xác:** Lộ trình tìm được không chỉ dựa trên khoảng cách vật lý đơn thuần mà còn có khả năng tự động tránh các khu vực có mật độ giao thông cao. Điều này đạt được nhờ vào hàm chi phí linh hoạt đã định nghĩa:

$$Weight = Length \times (1 + \alpha \times JamLevel) \quad (13)$$

Giúp tối ưu hóa thời gian thực thay vì chỉ tối ưu hóa quãng đường.

- Hiệu năng vận hành:** Thuật toán  $A^*$  cho thấy tốc độ xử lý nhanh chóng ngay cả trên quy mô đồ thị lớn (hơn 10,000 nodes). Việc sử dụng hàm Heuristic Euclid giúp thu hẹp không gian tìm kiếm, đảm bảo thời gian phản hồi tức thì – một yếu tố sống còn trong điều kiện khẩn cấp của y tế.
- Tính thực tiễn cao:** Với 114 tọa độ trung gian được trích xuất, lộ trình mô phỏng bám sát hoàn toàn theo các đoạn đường cong, vòng xuyên và nút giao phức tạp. Điều này chứng minh cấu trúc dữ liệu đồ thị của hệ thống đã biểu diễn rất tốt hạ tầng giao thông thực tế.

### 4.5 Hệ thống tích hợp

#### 4.5.1 Cấu hình kịch bản

Hệ thống tiến hành chạy thử nghiệm toàn trình (end-to-end) với cấu hình môi trường giả định và danh sách bệnh nhân như sau:

- Thời gian trong ngày (`time_of_day`): "Night"
- Ngày trong tuần (`day_of_week`): "Weekday"
- Thời tiết (`weather_condition`): "Rain"

| ID | Tọa độ                                  | Triệu chứng                                    |
|----|-----------------------------------------|------------------------------------------------|
| 1  | (28.60833561942784, 77.10891723632814)  | frontal headache, dizziness                    |
| 2  | (28.566733135974424, 77.17552185058595) | fainting, feeling ill, vomiting blood          |
| 3  | (28.615267765944424, 77.22633361816408) | sore throat, hoarse voice, coughing up sputum  |
| 4  | (28.557083832723716, 77.08660125732422) | bleeding from eye, bleeding from ear, fainting |

Table 4: Danh sách bệnh nhân đầu vào của hệ thống tích hợp

#### 4.5.2 Kết quả thực thi từng giai đoạn

*Giai đoạn 1:* Dự báo giao thông

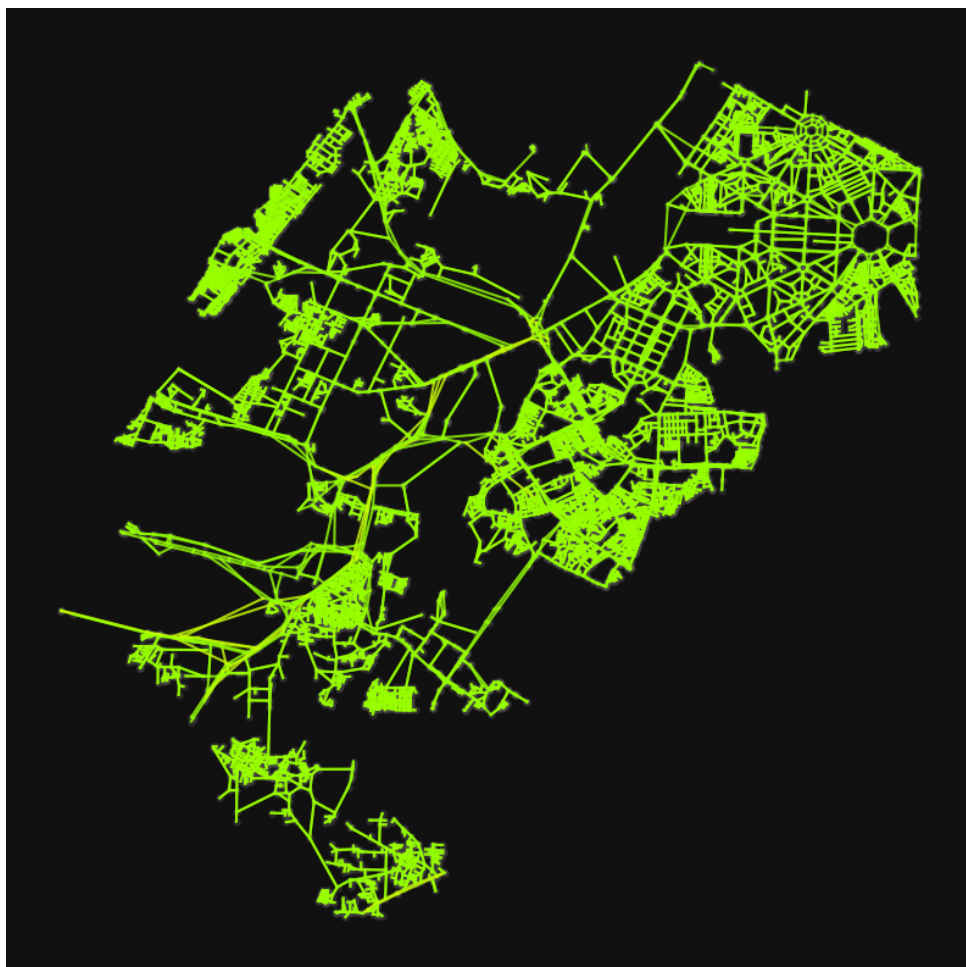


Figure 6: Bản đồ nhiệt phản ánh dự báo tình trạng giao thông.

*Giai đoạn 2:* Chẩn đoán và phân tầng ưu tiên

| ID | Triệu chứng                                    | Chẩn đoán                                    | Ưu tiên | Đường đi   | Được chọn |
|----|------------------------------------------------|----------------------------------------------|---------|------------|-----------|
| 1  | frontal headache, dizziness                    | benign paroxysmal positional vertical (bppv) |         |            |           |
| 2  | fainting, feeling ill, vomiting blood          | hypovolemia                                  | *       | 6790.76 m  | *         |
| 3  | sore throat, hoarse voice, coughing up sputum  | head and neck cancer                         |         |            |           |
| 4  | bleeding from eye, bleeding from ear, fainting | hypovolemia                                  | *       | 15316.85 m |           |

Table 5: Bảng tổng hợp kết quả chẩn đoán, xếp hạng ưu tiên và tính toán lộ trình

*Giai đoạn 3:* Kết xuất lộ trình điều phối cuối cùng

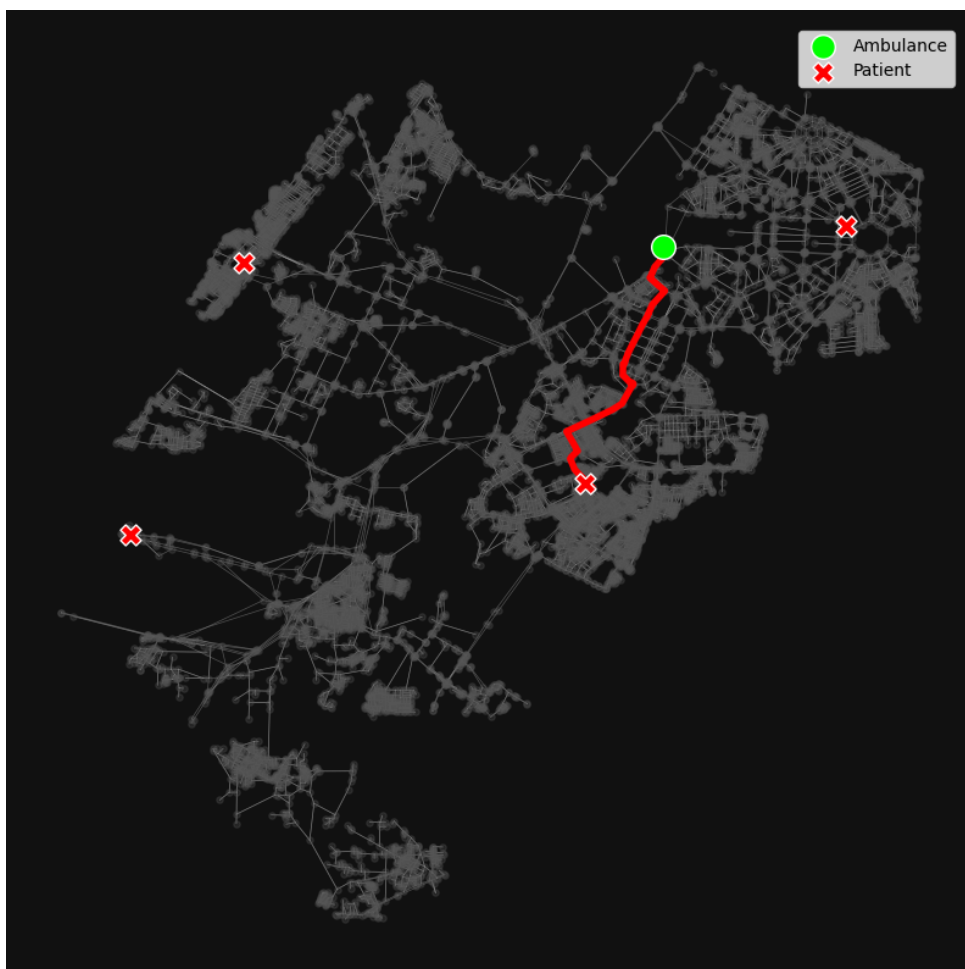


Figure 7: Lộ trình tối ưu hướng tới bệnh nhân được chọn (ID 2).

#### 4.5.3 Tổng kết chu trình điều phối

Hệ thống đã vận hành thành công pipeline tích hợp từ khâu chẩn đoán đến ra quyết định điều phối. Mô hình đã xác định chính xác bệnh lý khẩn cấp (*hypovolemia*) cho bệnh nhân ID 2 và ID 4 dựa trên các triệu chứng nghiêm trọng như *fainting* và *vomiting blood*.

Trong tình huống có nhiều bệnh nhân cùng đạt mức ưu tiên cao nhất, hệ thống đã chứng minh được khả năng giải quyết xung đột khi chuyển tiếp dữ liệu sang mô-đun tìm đường. Dựa trên sự kết hợp giữa thuật toán  $A^*$  và hàm tính toán cự ly vật lý, hệ thống đã lựa chọn bệnh nhân ID 2 làm đích đến cuối cùng do sở hữu lộ trình di chuyển ngắn hơn đáng kể (6790.76 m so với 15316.85 m của ID 4).

Mặc dù dự báo giao thông từ Mạng Bayes chưa tạo ra sự phân hóa quá rệt về mật độ do giới hạn của bộ dữ liệu huấn luyện, kịch bản thực nghiệm này đã minh chứng hoàn hảo cho tính toàn vẹn, tính logic và khả năng kết nối mượt mà giữa các mô-đun AI độc lập thành một hệ thống ra quyết định thống nhất.

## 5 Kết luận

Đề tài "Hệ thống điều phối xe cứu thương" đã giải quyết thành công bài toán tối ưu hóa nguồn lực y tế khẩn cấp bằng cách xây dựng một đường ống tích hợp toàn diện bốn mô-đun AI:

- **Học máy (Naive Bayes):** Chẩn đoán bệnh lý đạt độ chính xác 84.53%, với tốc độ suy diễn nhanh hơn 15 lần và tiết kiệm 39% RAM so với thư viện chuẩn.
- **Suy luận logic (pyDataLog):** Tự động phân tầng mức độ khẩn cấp chính xác, mô phỏng hoàn hảo tư duy y khoa.

- **Mạng Bayes:** Chuyển hóa các yếu tố môi trường bất định (thời tiết, thời gian) thành tham số rủi ro kẹt xe mang tính định lượng.
- **Tìm kiếm tối ưu ( $A^*$ ):** Tính toán nhanh chóng lộ trình di chuyển ngắn nhất trên không gian đồ thị giao thông thực tế (Delhi, India).

Kết quả thực nghiệm cho thấy hệ thống có thể vận hành một cách liên mạch theo, từ chẩn đoán, phân tầng ưu tiên đến tìm đường, qua đó hỗ trợ quyết định điều phối mà không cần can thiệp thủ công.

### Hướng phát triển tương lai

Mô hình có thể tiếp tục được nâng cấp bằng cách tích hợp dữ liệu giao thông thời gian thực (Real-time Traffic API). Đồng thời, hệ thống có thể mở rộng thành một Hệ Đa tác tử (Multi-agent System) để quản lý luồng điều phối của nhiều xe cứu thương cùng lúc, hướng tới ứng dụng thực tiễn trong các đô thị thông minh.

## Tài liệu tham khảo

- [1] Ankur Ankan, & Johannes Textor (2024). *pgmpy: A Python Toolkit for Bayesian Networks*. Journal of Machine Learning Research, 25(265), 1–8.
- [2] Boeing, G. (2025). *Modeling and Analyzing Urban Networks and Amenities with OSMnx*. Geographical Analysis, 57(4), 567–577. doi:10.1111/gean.70009.
- [3] Buitinck, L. et al. (2013). *API design for machine learning software: experiences from the scikit-learn project*. ECML PKDD Workshop: Languages for Data Mining and Machine Learning. 108–122.
- [4] Carbonnelle, P. *pyDatalog: Datalog logic programming in Python*. Google Sites documentation page. URL: <https://sites.google.com/site/pydatalog/home>.
- [5] Dhivyesh R K. *Disease-Symptom Dataset*. Kaggle dataset. URL: <https://www.kaggle.com/datasets/dhivyeshrk/diseases-and-symptoms-dataset>.
- [6] Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). *Exploring network structure, dynamics, and function using NetworkX*. In G. Varoquaux, T. Vaught, and J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference (SciPy2008)*. Pasadena, CA, USA, pp. 11–15.
- [7] Harris, C. R., Millman, K. J., van der Walt, S. J. et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362. doi:10.1038/s41586-020-2649-2.
- [8] Tanzeela Aftab. *Delhi Traffic Patterns Dataset*. Kaggle dataset. URL: <https://www.kaggle.com/datasets/guriya79/understanding-delhi-traffic-patterns>.